
BINARY-JEPA : HACHAGE SÉMANTIQUE DE BINAIRES PAR PRÉDICTION LATENTE AUTO-SUPERVISÉE SUR BAG-OF-PATHS VEX

WORKING PAPER

 **Ryan Bouchou***

ING3 Cybersécurité
CY Tech
64000 Pau, France

`ryan.bouchou@etu.cyu.fr`

 **Ruben Petteng-Ngongang**

ING3 Cybersécurité
CY Tech
64000 Pau, France

`ruben.pentteng-ngongang@etu.cyu.fr`



`iRyann/binary-jepa`



`rubenanko/binary-jepa`

28 mars 2026

ABSTRACT

Face à la prolifération des malwares polymorphes, les signatures cryptographiques classiques telles que MD5 ou SHA-256 sont structurellement obsolètes : un seul bit modifié suffit à invalider toute comparaison exacte. Ce rapport technique présente **Binary-JEPA**, une architecture de hachage sémantique de binaires ELF64 fondée sur l'apprentissage auto-supervisé, conçue pour produire une empreinte stable face aux transformations compilatoires et, à terme, aux techniques d'obfuscation modernes.

Notre approche repose sur trois contributions articulées. Premièrement, un pipeline d'extraction statique traduisant le code binaire en représentation intermédiaire VEX IR via `angr`, puis modélisant la topologie du programme sous forme d'un *Bag-of-Paths* — un multiensemble de chemins d'exécution canonicalisés en 371 tokens abstraits par une projection sémantique stricte Φ , s'affranchissant des artefacts compilatoires. Deuxièmement, une adaptation unidimensionnelle du cadre I-JEPA [1] à l'analyse de séquences symboliques : un encodeur convolutif léger est entraîné à prédire la représentation latente de positions masquées plutôt qu'à reconstruire le token exact, forçant l'émergence d'une sémantique comportementale sans supervision. Troisièmement, un paquet Python distribuable (`elf64ctx`) encapsulant le pipeline complet avec téléchargement automatique des artefacts depuis HuggingFace Hub.

Une preuve de concept qualitative dénote deux propriétés attendues de l'espace latent : une invariance partielle à la compilation pour les paires positives, et une séparabilité inter-programmes pour les paires négatives, avec un effet super-additif lorsque les deux sources de divergence se cumulent. L'analyse de trois runs d'entraînement révèle que la stratégie de masquage constitue le facteur de convergence dominant : seul le masquage aléatoire produit un plateau stable à $\mathcal{L} \approx 0.030$ sur 10 époques.

Ce rapport constitue un *working paper* : les hypothèses de résilience face aux obfuscations restent à valider empiriquement, et l'évaluation quantitative fait l'objet des travaux en cours.

Keywords Malware Analysis · Semantic Hashing · Self-Supervised Learning · VEX IR · Bag-of-Paths (sac de chemins) · Joint-Embedding Predictive Architecture (JEPA)

*<https://www.ryanbouchou.fr>, ryanbouchou.pro@gmail.com

1 Introduction

Le paysage de la cybersécurité fait face à une croissance fulgurante et continue des menaces, aujourd’hui accélérée par le biais des modèles génératifs tels que Large Language Models (LLMs), et les automatisations agentiques diverses. Les éditeurs de sécurité détectent actuellement environ 350 000 nouveaux programmes malveillants ou applications indésirables chaque jour [2,3]. Sur la seule année 2024, approximativement huit millions de nouveaux échantillons de malwares ont été identifiés [3]. Cette prolifération est par ailleurs exacerbée par l’expansion des objets connectés (Internet of Things (IoT)) et des systèmes embarqués, qui offrent une surface d’attaque massive pour de nouvelles variantes de botnets (telles que Mirai ou Torii) et autres codes malveillants ciblant de multiples architectures [4].

Face à ce volume gigantesque et à l’évolution constante des techniques d’attaque, l’analyse manuelle des exécutables par des experts en rétro-ingénierie est devenue une tâche trop lente, coûteuse et fastidieuse [5,6]. L’automatisation de l’analyse statique s’impose donc comme une nécessité absolue pour identifier, classer et regrouper rapidement ces menaces. Pour faire face à ce défi, la communauté scientifique et l’industrie se tournent massivement vers le développement d’outils automatisés propulsés par l’apprentissage automatique² (Machine Learning (ML) et Deep Learning (DL)) [5].

Cependant, bien que l’analyse statique soit essentielle, se baser uniquement sur le code assembleur brut ou sur la topologie inhérente du programme présente de graves lacunes face aux mutations du code :

La fragilité de l’assembleur brut Un même code source compilé génère des séquences d’instructions radicalement différentes en fonction de l’architecture matérielle ciblée (x86, ARM, MIPS), du compilateur utilisé (par exemple GCC ou Clang) et du niveau d’optimisation choisi (de -O0 à -O3, ou -Os) [7,8]. Le processus de compilation entraîne naturellement une perte d’informations sémantiques (comme les noms de variables ou de fonctions) et modifie profondément la séquence linéaire des instructions via des optimisations telles que le fonction inlining ou la réallocation de registres [7,8]. Une comparaison purement syntaxique des octets ou de l’assembleur échoue donc rapidement à identifier des similarités.

La fragilité du graphe de flux de contrôle (CFG) Pour pallier les limites de l’assembleur linéaire, l’analyse s’est historiquement tournée vers les CFGs, qui modélisent les chemins d’exécution du programme. Néanmoins, les CFGs sont extrêmement sensibles aux techniques d’obfuscation, couramment utilisées par les malwares, qui déforment intentionnellement la structure du code sans en altérer la sémantique [3]. Par exemple, l’insertion de Bogus Control Flow (BCF) ajoute des blocs de base et des arêtes factices au graphe. De même, l’aplatissement du flux de contrôle (Control Flow Flattening (FLA)) détruit la hiérarchie logique du programme pour forcer toutes les exécutions à passer par un bloc répartiteur central [3]. Ces techniques modifient tellement la matrice d’adjacence et la topologie du graphe que les algorithmes de correspondance structurelle, ainsi que les Graph Neural Networks (GNNs) classiques, deviennent incapables de reconnaître des fonctions pourtant sémantiquement identiques [3].

Pour surmonter ces limites inhérentes aux représentations syntaxiques et topologiques, cet article propose une nouvelle approche de détection de similarité de code binaire (Binary Code Similarity Detection (BCSD)) basée sur l’apprentissage auto-supervisé. Plus précisément, nos contributions principales (*claims*) s’articulent autour des quatre axes suivants :

- **Modélisation Sémantique Robuste (VEX & Bag-of-Paths)** : Nous proposons un pipeline d’extraction qui traduit le code binaire en une représentation intermédiaire universelle (VEX Intermediate Representation (VEX IR)) afin de s’abstraire de l’architecture matérielle. La topologie du programme est modélisée sous la forme d’un Bag-of-Paths, court-circuitant le bruit système (Procedure Linkage Table (PLT)/C standard library (libc)) et offrant une résilience naturelle face aux obfuscations modifiant le graphe de contrôle (CFG).
- **Architecture Prédictive Latente (I-JEPA 1D)** : Contrairement aux approches génératives traditionnelles qui tentent de reconstruire le code exact, nous adaptons l’architecture Improved Joint-Embedding Predictive Architecture (I-JEPA) au traitement de séquences 1D. En forçant un réseau convolutif à prédire la représentation latente des instructions masquées, le modèle apprend la sémantique profonde du chemin d’exécution tout en ignorant structurellement le code mort (junk code).
- **hachage sémantique et Évaluation** : Nous introduisons une méthode de hachage sémantique efficace exploitant l’encodeur pré-entraîné.

2. Des précisions sur l’usage de telles solutions sont données en Annexe A.2

- **Ingénierie Machine Learning Operations (MLOps) et Éco-responsabilité** : Nous fournissons une implémentation de niveau industriel (chargement de données *Just-In-Time*, entraînement distribué en précision mixte) intégrant un suivi rigoureux de l’empreinte carbone (Green AI), démontrant la viabilité de l’approche pour des déploiements à grande échelle.

2 État de l’Art

La détection de similarité de code binaire (BCSD - *Binary Code Similarity Detection*) a connu une évolution paradigmatique majeure au cours des deux dernières décennies, passant de la simple vérification d’intégrité à l’extraction de sémantique latente par réseaux de neurones profonds. Cette section passe en revue les différentes approches historiques et contemporaines, tout en soulignant leurs limites respectives face aux techniques modernes d’obfuscation.

2.1 Hachage Cryptographique et Méthodes Syntaxiques

Historiquement, la détection de malwares reposait sur des signatures cryptographiques strictes telles que MD5 ou SHA-256. Cependant, ces fonctions de hachage souffrent d’un effet d’avalanche absolu : la modification d’un seul bit (par exemple, par l’ajout d’une instruction NOP) altère totalement l’empreinte finale. Pour pallier cette fragilité, la communauté s’est tournée vers le hachage partiel (*Fuzzy Hashing*). Des algorithmes pionniers tels que *ssdeep* [9], qui exploite le hachage par morceaux déclenché par le contexte, ou TLSH (*Trend Micro Locality Sensitive Hash*) [10], ont été introduits pour tolérer des variations mineures au niveau de la séquence d’octets. Bien que plus robustes que les hachages cryptographiques, ces méthodes demeurent fondamentalement syntaxiques. Elles échouent drastiquement face aux obfuscateurs automatisés qui préservent la sémantique mais modifient l’ordre des instructions (réallocation de registres, *instruction substitution*).

2.2 Modèles de Langage et Transformers appliqués au Code Binaire

Face aux limites de l’analyse purement syntaxique, l’émergence du traitement du langage naturel (NLP) a inspiré la modélisation des séquences d’instructions comme un langage à part entière. Les architectures basées sur les Transformers ont récemment dominé ce champ de recherche. Le modèle BinBert [11] a démontré la viabilité de l’adaptation de l’architecture BERT pour comprendre le contexte d’exécution du code binaire. Pour résoudre les problèmes liés aux sauts conditionnels qui fragmentent la séquence, jTrans [12] a introduit un Transformer "Jump-Aware", capable d’intégrer explicitement les cibles de saut dans son mécanisme d’attention. Plus récemment, kTrans [13] a raffiné cette approche en injectant des connaissances préalables sur les architectures de jeux d’instructions (ISA), traitant l’assembleur non pas comme un texte brut, mais comme un système logique fortement contraint. Si ces modèles offrent une excellente résolution de la polysémie instructionnelle, le mécanisme d’auto-attention inhérent aux Transformers souffre d’une complexité computationnelle quadratique ($O(N^2)$), les rendant souvent impraticables pour l’analyse en temps réel de fonctions très volumineuses.

2.3 Réseaux de Graphes (GNN) et Résilience Topologique

En parallèle des modèles séquentiels, une large part de la recherche s’est concentrée sur la représentation du programme sous la forme d’un Graphe de Flux de Contrôle (CFG), traité via des Réseaux de Neurones sur Graphes (GNN). Cette approche est cependant très vulnérable aux attaques altérant la topologie, telles que l’aplatissement du flux de contrôle (*Control Flow Flattening* - FLA) ou l’insertion de faux blocs de base (*Bogus Control Flow* - BCF). Pour obtenir un invariant topologique robuste, de récents travaux comme ORCAS [14] ont proposé le Graphe Sémantique Amélioré par la Dominance (DESG). En se basant sur les relations de dominance (où un nœud domine un autre si tout chemin d’exécution vers le second passe par le premier) couplées aux chaînes de dépendance de données, ORCAS [14] parvient à abstraire la structure pour résister à des déformations agressives du CFG. Néanmoins, si l’extraction d’un Graphe de Flux de Contrôle (CFG) topologique de base est aujourd’hui optimisée, l’enrichissement sémantique de ces structures (tel que le calcul systématique des arbres de dominance) et l’inférence via GNN imposent une lourde charge computationnelle. Par ailleurs, d’un point de vue d’ingénierie matérielle (MLOps), les graphes posent des défis intrinsèques majeurs. Contrairement aux représentations séquentielles qui exploitent des opérations denses (GEMM) hautement optimisées sur accélérateurs matériels, les graphes requièrent des multiplications de matrices creuses (SpMM) et présentent des accès mémoire irréguliers [15]. De plus, la mise en lots (*batching*) de

graphes de tailles variables impose des structures de données complexes, comme l’agrégation en matrices diagonales par blocs [16], freinant considérablement leur passage à l’échelle sur des millions d’échantillons comparativement aux tenseurs 1D purs.

2.4 Optimisation du Hachage Profond et Discrétisation

L’objectif ultime d’un système BCSD à grande échelle est de produire un hachage binaire (une suite de 0 et de 1) permettant des recherches extrêmement rapides via des opérations CPU bas niveau (XOR, POPCOUNT). Cependant, la transformation d’un vecteur continu (issu d’un réseau de neurones) en valeurs discrètes pose un défi mathématique majeur : la fonction signe est non-différentiable, ce qui bloque la rétropropagation du gradient.

Pour contourner ce problème, la communauté utilise des estimateurs heuristiques comme le *Straight-Through Estimator* (STE) [17], qui permet au gradient de traverser l’opération de discrétisation de manière quasi-transparente lors de la phase d’entraînement. Afin d’assurer que la distance de Hamming dans l’espace binaire reflète fidèlement la proximité sémantique originelle, des fonctions de perte avancées ont été développées. OrthoHash [18] optimise l’alignement géométrique en maximisant la similarité cosinus entre les vecteurs continus et des cibles orthogonales prédéfinies sur l’hypercube binaire. Dans une optique complémentaire de filtrage du bruit (essentiel face à l’obfuscation), des approches basées sur la théorie de l’information comme le *Contrastive Information Bottleneck* (CIBHash) [19] cherchent à compresser la représentation pour ne conserver que le signal sémantique pur dans l’empreinte finale.

Bilan et Positionnement Malgré des avancées significatives, l’état de l’art met en lumière un compromis récurrent entre la finesse sémantique (Transformers, lents et coûteux), la robustesse structurelle (GNN, complexes à extraire), et l’efficacité à l’inférence. Le présent papier se positionne à l’intersection de ces contraintes en proposant une architecture inédite : l’utilisation d’un modèle prédictif latent (I-JEPA) couplé à une représentation en *Bag-of-Paths*. Cette synergie permet de conserver les avantages de la séquence et de la topologie, sans subir les limites computationnelles des graphes et de l’auto-attention quadratique.

3 Méthodologie

3.1 Intentions

De la syntaxe, à l’embedding, la sémantologie appliquée à l’informatique Du programme en tant que tel, comme fichier ELF, à la sémantique instillée par l’auteur de ce dernier, et jusqu’à la représentation qu’il en est donnée dans un espace approprié pour le Machine Learning, la substance manipulée se voit constamment remise par ces différentes *représentations*. Derrière cet agrégat informationnel projeté dans des espaces divers se pose la question de la perte informationnelle — et donc réciproquement de la conservation de celle-ci —, et de la fidélité à l’intention première matérialisée par le programme. Ces interrogations sont évidemment au fondement de l’embedding de code binaire, ou l’apprentissage de représentations vectorielles pour l’analyse binaire, visant à transformer des séquences d’instructions non structurées en vecteurs numériques denses de haute dimension capturant leur sémantique.

L’émergence latente : au-delà de l’embedding explicite Contrairement à la majorité des travaux de l’état de l’art qui traitent l’embedding comme un objectif premier et explicite — souvent contraint par des tâches de reconstruction syntaxique (Masked Language Modeling) ou d’isomorphisme de graphes —, notre approche opère un renversement de paradigme. En nous appuyant sur l’architecture I-JEPA [1], nous ne forçons pas la création d’un espace vectoriel par mimétisme syntaxique. L’architecture apprend à prédire la *représentation latente* d’un bloc de code manquant en se basant sur son contexte. Dès lors, pour minimiser son erreur de prédiction, le modèle est mathématiquement contraint d’ignorer le bruit stochastique (les variations d’obfuscation, le code mort) pour ne capter que l’essence comportementale du programme. L’embedding sémantique n’est plus l’artefact direct d’une fonction de hachage ; il jaillit de façon sous-jacente comme la seule structure topologique capable de lier le contexte à son évolution. L’intention première du programme se retrouve ainsi cristallisée, non pas parce qu’on a cherché à l’encoder, mais parce qu’elle était la condition sine qua non pour la prédire.

Actionnabilité Naturellement, même si l’intention est louable, elle n’en demeure pas moins sujette à de multiples défis. Car s’il nous faut révéler les invariants sémantiques d’une part — c’est-à-dire ce qui

distingue fondamentalement un algorithme d'un autre —, il nous faut aussi et surtout travailler dans un *espace de généralisation*. Autrement dit, il s'agit de proposer au modèle les données sous un format susceptible de permettre l'apprentissage de cette compréhension ontologique, tout en ne se polarisant pas sur les singularités propres à la compilation, à l'environnement, ou aux pratiques d'obfuscation.

De plus, il serait irréaliste de prétendre capturer l'exhaustivité du comportement algorithmique d'un programme sur la seule base de son code statique. Néanmoins, cette contrainte de l'analyse statique³ est inhérente à notre démarche, car elle constitue l'essence même de la notion de hachage de fichier. Ce paradigme impose donc un double défi :

1. La représentation des données doit être suffisamment générique pour généraliser, sans pour autant perdre l'ipséité nécessaire à la discrimination lors de l'entraînement.
2. L'analyse doit s'opérer de manière purement statique, sans exécution du binaire.

3.2 Jeu de Données

3.2.1 Choix du corpus et justification

Entraîner un modèle auto-supervisé requiert un corpus où les invariants sémantiques sont connus et contrôlables. Nous optons pour une approche de *compilation croisée contrôlée* : plutôt que d'utiliser des malwares réels (corpus bruités, déséquilibrés, soumis à des contraintes légales), nous construisons un jeu de données synthétique à partir de la suite **GNU Coreutils v9.10** [20], un ensemble de 106 utilitaires POSIX au comportement sémantique parfaitement documenté.

Ce choix repose sur une propriété fondamentale : deux compilations différentes d'un même programme *partagent la même sémantique* malgré des différences syntaxiques profondes (ordonnancement des blocs, choix des registres, inlining, déroulage de boucles). Le modèle est ainsi contraint d'apprendre des représentations *invariantes à la compilation* — exactement la propriété recherchée pour la détection de variants de malwares.

3.2.2 Génération du corpus

Le pipeline de génération est entièrement automatisé via `generate_data.sh`. Les sources sont téléchargées depuis le dépôt officiel de GNU Coreutils, puis chaque binaire est compilé selon la grille de variantes définie par le produit cartésien :

$$\mathcal{C} \times \mathcal{O} = \{\text{gcc}, \text{clang}\} \times \{-O0, -O1, -O2, -O3, -Os\}.$$

Chaque binaire $b \in \mathcal{B}$ (avec $|\mathcal{B}| = 106$) est donc compilé en $|\mathcal{C}| \times |\mathcal{O}| = 10$ variants distincts, pour un total de $106 \times 10 = 1\,060$ fichiers ELF. Deux précautions sont systématiquement appliquées :

- **Stripping complet** (`-strip-all`) : les symboles de débogage, les tables de noms et les sections `.debug_*` sont supprimés. Cela simule les conditions réelles d'analyse de malwares, où les informations symboliques sont absentes.
- **Désactivation de la localisation** (`-disable-nls`) : les chaînes de messages localisés sont retirées, réduisant la variabilité des données statiques non-sémantiques dans les sections `.rodata`.

3.2.3 Structure du corpus et statistiques

Le Tableau 1 résume les caractéristiques du corpus après application du pipeline d'extraction (Section 1).

La Figure 1 illustre la structure de la grille de compilation et la relation de *co-sémantique* entre variants d'un même binaire.

3.2.4 Limitations et perspectives

Ce corpus présente deux limitations majeures qu'il convient d'anticiper pour la généralisation à un contexte de détection de malwares réels.

Premièrement, GNU Coreutils est un corpus de *logiciels bénins*, compilé dans des conditions nominales. Il ne contient ni obfuscation, ni packing, ni techniques d'anti-analyse. Le modèle entraîné sur ce corpus apprend des représentations de code légitime, ce qui constitue une étape préalable à un entraînement par transfert sur des malwares (*fine-tuning*).

3. Pour un rappel des fondements de l'analyse statique, le lecteur pourra se référer à l'Annexe A.1

TABLE 1 – Statistiques du corpus d’entraînement (GNU Coreutils 9.10, après extraction du Bag-of-Paths).

Caractéristique	Valeur
Source	GNU Coreutils 9.10
Nombre de programmes distincts $ \mathcal{B} $	106
Compilateurs $ \mathcal{C} $	2 (gcc, clang)
Niveaux d’optimisation $ \mathcal{O} $	5 (-O0 à -Os)
Fichiers ELF totaux	1 060
Symboles conservés	Aucun (strip -strip-all)
Longueur max de séquence L	22000 tokens
Taille du vocabulaire $ \mathcal{V} $	371 tokens

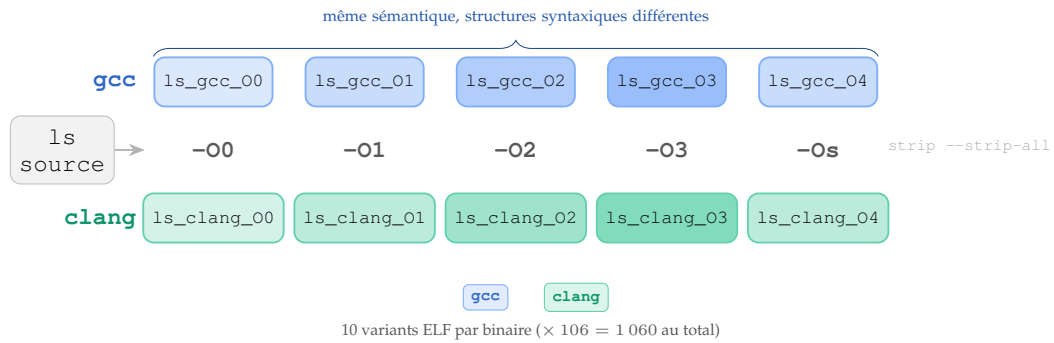


FIGURE 1 – Grille de compilation pour un binaire représentatif (ls). Chaque cellule est un fichier ELF strippé indépendant. Les 10 variants d’un même programme partagent une sémantique identique mais des représentations binaires structurellement distinctes, offrant un signal d’entraînement riche pour l’invariance à la compilation.

Deuxièmement, l’ensemble des binaires provient d’une unique base de code (un seul projet, un seul langage source, une seule architecture cible x86-64). La diversité des patterns sémantiques est donc plus limitée que dans un corpus multi-famille de malwares. Une extension naturelle consiste à incorporer d’autres projets open-source de grande envergure (noyau Linux, OpenSSL, FFmpeg) afin d’élargir la couverture sémantique du vocabulaire VEX appris.

3.3 Extraction et Représentation des Données

La première étape de notre architecture consiste à transformer le code binaire brut en une représentation mathématique discrète, apte à être ingérée par un modèle de Deep Learning, tout en filtrant le bruit syntaxique induit par la compilation. Pour ce faire, nous proposons un pipeline d’extraction statique en trois phases : le levage sémantique (*lifting*), l’échantillonnage topologique en *Bag-of-Paths*, et la projection par canonicalisation.

3.3.1 Levage Sémantique et Extraction du CFG

Étant donné un fichier binaire exécutable $\mathcal{B}$, nous utilisons le framework d’analyse binaire *angr* pour désassembler et lever les instructions assembleur (x86, ARM, etc.) vers la représentation intermédiaire VEX IR. Cette étape permet de s’abstraire totalement des spécificités du matériel (registres physiques, drapeaux d’état) pour se concentrer sur les macro-opérations logiques. Pour chaque fonction $f \in \mathcal{B}$ (à l’exclusion des appels systèmes et des fonctions de bibliothèques externes résolues dynamiquement), nous extrayons son Graphe de Flux de Contrôle (CFG) intra-procédural, noté $G_f = (V_f, E_f)$, où V_f représente l’ensemble des blocs de base (*basic blocks*) et E_f les transitions de contrôle (sauts, branchements conditionnels).

3.3.2 Topologie en Bag-of-Paths (BoP)

Les approches classiques traitent G_f soit de manière aplatie, soit via des réseaux de neurones sur graphes (GNN). Pour garantir une résilience face aux obfuscations modifiant agressivement la matrice d'adjacence (comme l'aplatissement du flux de contrôle - FLA), nous décomposons la topologie du programme en un ensemble de chemins d'exécution indépendants.

Nous définissons un chemin p de longueur L comme une séquence de blocs de base $p = (v_1, v_2, \dots, v_L)$ telle que $\forall i \in \{1, \dots, L-1\}, (v_i, v_{i+1}) \in E_f$. Afin de pallier l'explosion combinatoire des chemins (*path explosion*), inhérente à l'analyse statique, nous implémentons un algorithme de recherche en profondeur (DFS) aléatoire itératif. À partir du bloc d'entrée de la fonction v_{entry} , nous échantillonnons un sous-ensemble de chemins maximaux avec une longueur maximale fixée à $L_{max} = 50$, et limitons le nombre total de chemins extraits à $N_{max} = 500$. Ainsi, chaque fonction f est représentée par un multiensemble (ou *Bag*) de chemins :

$$\mathcal{B}_f = \{p_1, p_2, \dots, p_N\} \quad \text{avec} \quad N \leq N_{max} \quad (1)$$

3.3.3 Canonicalisation VEX et Résolution des API

La traduction directe du VEX IR produit un vocabulaire extrêmement vaste (adresses mémoires hardcodées, noms de registres temporaires τ_1, τ_2), conduisant irrémédiablement au problème des mots inconnus (OOV - *Out-Of-Vocabulary*). Nous introduisons donc une fonction de projection sémantique stricte $\Phi : \mathcal{V}_{VEX} \rightarrow \mathcal{T}$ qui mappe chaque déclaration VEX vers un token abstrait t issu d'un dictionnaire fini $\mathcal{T}$.

L'opérateur Φ applique les transformations suivantes sur les opérandes et instructions :

- **Opérations et Mémoire** : Les écritures et lectures sont abstraites par l'intention de l'action : $Ist_Put \mapsto VEX_REG_WRITE, Iex_Load \mapsto VEX_LOAD$.
- **Opérateurs arithmétiques** : Les calculs sont réduits à leur classe opératoire principale (ex : $Iop_Sub32 \mapsto VEX_OP_SUB$).
- **Résolution des appels terminaux** : Si un bloc se termine par un saut vers la table de liaison (PLT) ou un appel système, l'intention est explicitement capturée dans la séquence finale via un token dédié (ex : $\langle API_PUTS \rangle$ ou $\langle SYSCALL_EXECVE \rangle$). Dans le cas contraire, le type de saut (*JumpKind*) est conservé (ex : JK_RET, JK_BORING).

Grâce à cette projection Φ , chaque chemin $p = (v_1, \dots, v_L)$ est aplati en une unique séquence de tokens 1D de longueur S , constituant l'entrée de notre modèle I-JEPA. Ce dictionnaire minimaliste garantit un espace d'encodage dense et force l'architecture neuronale à déduire la sémantique de l'algorithme à partir du contexte global de la séquence, plutôt qu'en mémorisant des adresses spécifiques.

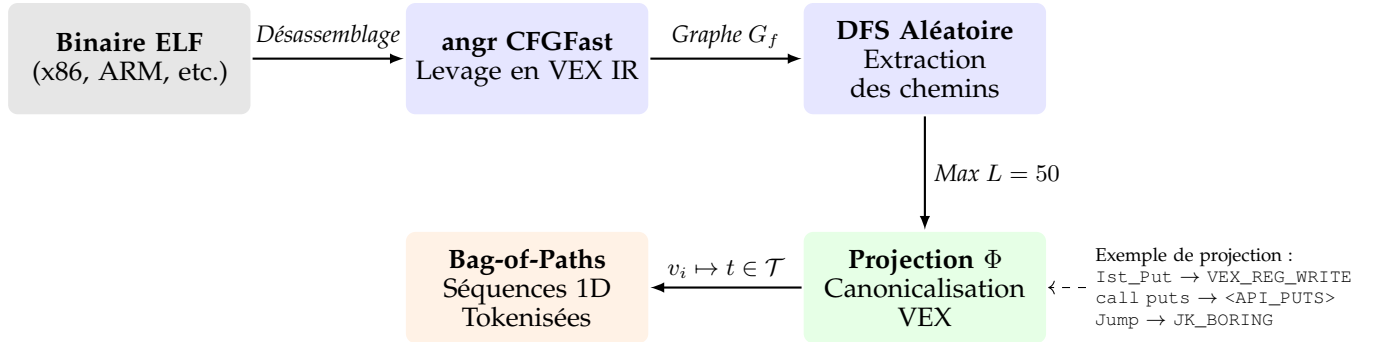


FIGURE 2 – Pipeline d'extraction statique et de projection sémantique générant les séquences du Bag-of-Paths.

3.3.4 Propriétés

Hypothèses

1. Modélisation offrant une résilience naturelle face aux obfuscations modifiant le graphe de contrôle (CFG).
2. Modélisation offrant une couverture sémantique dense et relativement uniforme au gré des transformations, notamment en vue de l'apprentissage (et donc au regard des particularités de ce dernier).

Pour l’heure, compte-tenu des contraintes inhérentes à la réalisation de ce projet de fin d’étude, les hypothèses formulées 3.3.4 sont postulées, dans l’attente d’une validation quantitative, une fois l’architecture globale stabilisée.

3.3.5 Robustesse Opérationnelle MLOps

L’analyse statique de malwares est un processus notoirement instable : les techniques d’obfuscation avancées (comme UPX ou Themida) peuvent forcer les moteurs symboliques ou de levage dans des boucles infinies ou provoquer des épuisements de mémoire RAM (OOM - *Out-Of-Memory*). Pour garantir le passage à l’échelle sur des millions d’échantillons, notre pipeline est encapsulé dans une architecture multiprocessus robuste exploitant un isolement temporel strict (*hard-timeout* via signaux SIGTERM). De plus, un cache de blocs intra-fonction garantit qu’une adresse n’est soulevée en VEX qu’une seule fois, même si elle participe à des centaines de chemins, optimisant drastiquement le temps d’extraction global.

3.4 Architecture du Modèle Auto-Supervisé

3.4.1 Vue d’ensemble

Nous proposons une adaptation unidimensionnelle du cadre *Image-based Joint-Embedding Predictive Architecture* (I-JEPA, [1]) à l’analyse statique de binaires ELF. La représentation d’entrée est une *séquence de tokens VEX/API* de longueur fixe L , notée

$$\mathbf{x} = (x_1, x_2, \dots, x_L) \in \mathcal{V}^L,$$

où $\mathcal{V}$ est le vocabulaire discret de taille $|\mathcal{V}| = 371$ (voir Section 1). Le modèle comprend trois composants appris : un *encodeur de contexte* f_θ , un *encodeur cible* f_ξ (également appelé *professeur*), et un *prédicteur* g_ϕ . L’architecture globale est illustrée en Figure 3.

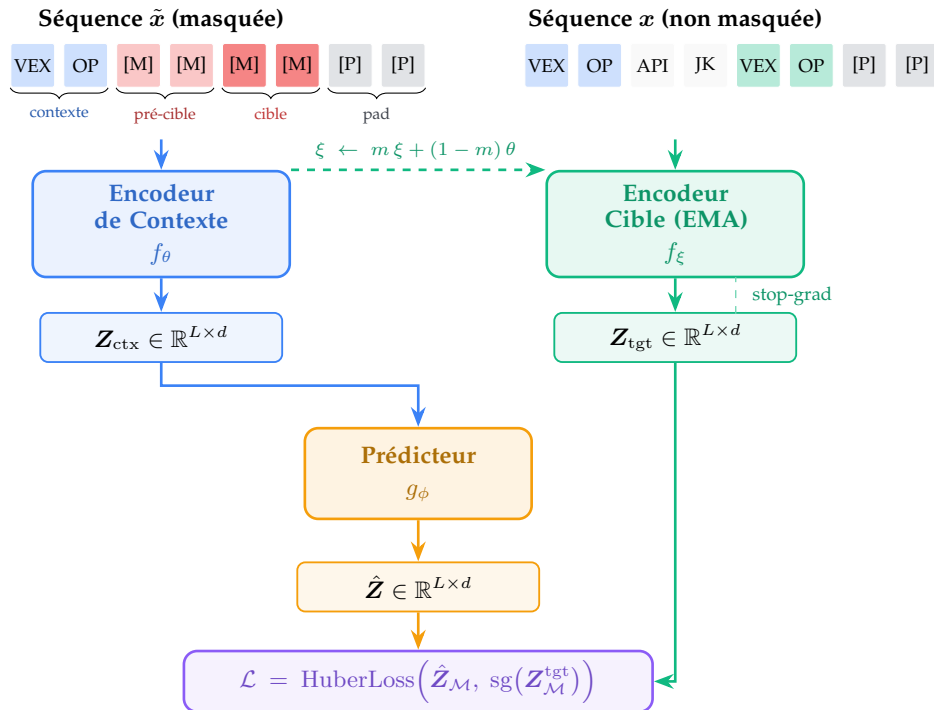


FIGURE 3 – Architecture I-JEPA 1D. La séquence masquée $\tilde{x}$ alimente l’encodeur de contexte f_θ (bleu); la séquence intacte x alimente l’encodeur cible f_ξ (vert), mis à jour exclusivement par EMA (flèche pointillée). Le prédicteur g_ϕ (ambre) produit $\hat{Z}$ depuis Z_{ctx} , et la perte de Huber est calculée uniquement sur la zone cible $\mathcal{M}$. Aucun gradient ne se propage à travers f_ξ (*stop-gradient*).

3.4.2 Représentation d’Entrée

Chaque chemin extrait par le pipeline (Section 2) est une liste de tokens symboliques issus de l’IR VEX et des points de sortie vers les API système. Les tokens sont convertis en indices entiers par le vocabulaire $\mathcal{V}$, puis les séquences sont tronquées à $L = 500$ tokens et complétées par le token spécial $\langle \text{PAD} \rangle$ (identifiant 0) pour atteindre la longueur fixe L . Formellement, pour un chemin p de longueur ℓ_p ,

$$x_i = \begin{cases} v(p_i) & \text{si } i \leq \ell_p, \\ 0 & \text{sinon (padding),} \end{cases}$$

où $v : \mathcal{V} \rightarrow \{0, \dots, |\mathcal{V}| - 1\}$ est la fonction d’encodage du vocabulaire.

3.4.3 Encodeur de Contexte f_θ

L’encodeur de contexte projette une séquence de tokens (partiellement masquée) dans un espace de représentation continu de dimension $d = 256$. Il est composé d’une couche d’embedding suivie d’un réseau de convolutions 1D causales.

Embedding. Les indices entiers sont d’abord projetés dans $\mathbb{R}^e$ via une matrice apprise $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times e}$, avec $e = 128$:

$$\mathbf{H}^{(0)} = \mathbf{E}[\mathbf{x}] \in \mathbb{R}^{L \times e}.$$

Réseau convolutif. Trois couches de convolution 1D sont appliquées successivement sur la dimension temporelle, selon le schéma $e \rightarrow h \rightarrow h \rightarrow d$ avec $h = 256$ et $d = 256$:

$$\mathbf{H}^{(1)} = \text{GELU}\left(\text{Conv1D}_{k=5}^{e \rightarrow h}\left(\mathbf{H}^{(0)}\right)\right), \quad (2)$$

$$\mathbf{H}^{(2)} = \text{GELU}\left(\text{Conv1D}_{k=5}^{h \rightarrow h}\left(\mathbf{H}^{(1)}\right)\right), \quad (3)$$

$$\mathbf{Z} = \text{Conv1D}_{k=3}^{h \rightarrow d}\left(\mathbf{H}^{(2)}\right) \in \mathbb{R}^{L \times d}. \quad (4)$$

Les convolutions utilisent un padding “same” ($p = \lfloor k/2 \rfloor$), de sorte que la longueur de la séquence est préservée à chaque étape. L’encodeur cible f_ξ partage la même architecture que f_θ mais possède ses propres paramètres, mis à jour exclusivement par EMA (Section 3.4.6).

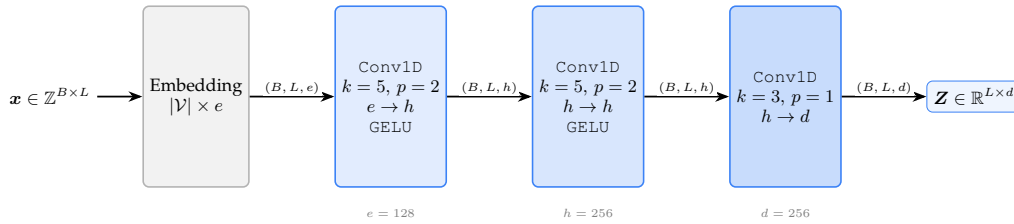


FIGURE 4 – Architecture de l’encodeur convolutif 1D f_θ (identique pour f_ξ). Les dimensions sont indiquées par batch B , longueur de séquence L , et canaux. Le padding *same* préserve L tout au long du réseau.

3.4.4 Prédicteur g_ϕ

Le prédicteur est un réseau entièrement connecté léger opérant en *position-wise* sur les représentations produites par f_θ . Pour chaque position $i \in \{1, \dots, L\}$:

$$\hat{z}_i = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 z_i^{\text{ctx}} + \mathbf{b}_1) + \mathbf{b}_2,$$

avec $\mathbf{W}_1 \in \mathbb{R}^{d_h \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times d_h}$, $d_h = 512$. En notation matricielle (sur le batch entier),

$$\hat{\mathbf{Z}} = g_\phi(\mathbf{Z}_{\text{ctx}}) \in \mathbb{R}^{B \times L \times d}.$$

Le prédicteur est intentionnellement peu profond : sa capacité réduite l’empêche de mémoriser la séquence d’entrée et force l’encodeur de contexte à produire des représentations sémantiquement riches [1].

3.4.5 Stratégie de Masquage

Contrairement aux approches de masquage uniforme (BERT, MAE), nous adoptons un masquage par *blocs contigus* tenant compte de la longueur réelle ℓ_i de chaque séquence dans le batch, définie par

$$\ell_i = |\{j : x_{i,j} \neq x_{\text{PAD}}\}| = \sum_{j=1}^L \mathbf{1}[x_{i,j} \neq 0].$$

La séquence réelle est partitionnée en trois blocs de taille approximativement égale. Deux masques booléens sont construits pour chaque exemple i du batch :

- **Masque d’entrée** $\mathcal{M}_i^{\text{in}}$ — positions $[\lfloor \ell_i/3 \rfloor, \ell_i)$ sont remplacées par $\langle \text{MASK} \rangle$ (ID 2) avant d’entrer dans f_θ . Cela cache les blocs 2 et 3 au contexte.
- **Masque de prédiction** $\mathcal{M}_i^{\text{pred}}$ — positions $[\lfloor 2\ell_i/3 \rfloor, \ell_i)$ constituent la *cible* : c’est sur ces positions uniquement que la perte est calculée.

Formellement, la séquence présentée à f_θ est

$$\tilde{x}_{i,j} = \begin{cases} x_{\text{MASK}} & \text{si } j \in \mathcal{M}_i^{\text{in}}, \\ x_{i,j} & \text{sinon.} \end{cases}$$

La Figure 5 illustre cette stratégie en tiers sur une séquence de longueur réelle arbitraire ℓ .

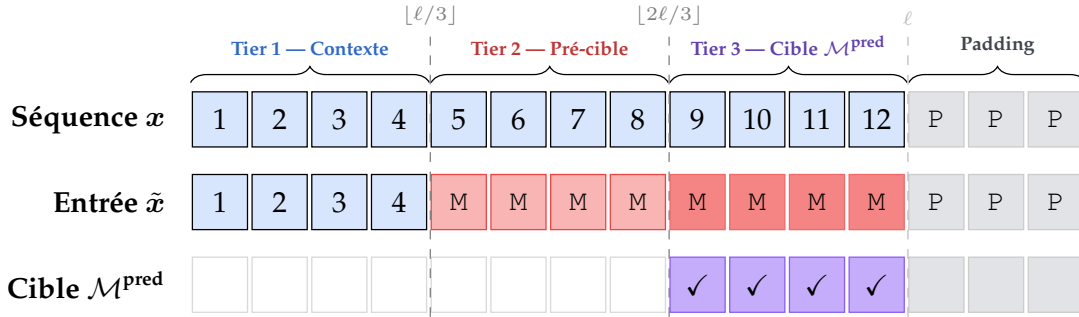


FIGURE 5 – Stratégie de masquage par tiers. La séquence réelle de longueur ℓ est divisée en trois blocs égaux. Le Tier 2 (pré-cible) et le Tier 3 (cible) sont remplacés par $\langle \text{MASK} \rangle$ dans l’entrée de f_θ . La perte est calculée uniquement sur le Tier 3 ($\mathcal{M}^{\text{pred}}$). Le padding ne contribue ni au masquage ni à la perte.

Cette conception assure que le modèle apprend à inférer la *continuation sémantique* d’un chemin d’exécution, ce qui est analogue à la capacité d’un analyste à anticiper quels appels système ou opérations mémoire suivront une séquence d’instructions observée. Elle diffère du masquage aléatoire (BERT) en forçant une dépendance *ordonnée* : le contexte est toujours antérieur à la cible, ce qui respecte la nature directionnelle d’un chemin de flot de contrôle.

3.4.6 Mise à jour par Moyenne Mobile Exponentielle

L’encodeur cible f_ξ n’est jamais optimisé directement par rétropropagation. Ses paramètres ξ sont mis à jour après chaque itération t selon la règle EMA :

$$\xi_t \leftarrow m \cdot \xi_{t-1} + (1 - m) \cdot \theta_t, \quad (5)$$

où $m \in (0, 1)$ est le momentum EMA, fixé à $m = 0,996$ dans nos expériences. L’initialisation est $\xi_0 = \theta_0$.

Ce mécanisme joue un rôle fondamental dans la stabilité de l’apprentissage : il agit comme un *stop-gradient* implicite sur la branche cible, empêchant l’effondrement de la représentation (*representation collapse*) qui surviendrait si les deux encodeurs évoluaient librement. L’encodeur cible produit ainsi des cibles *lissées dans le temps* qui guident l’encodeur de contexte vers des représentations cohérentes. Ce principe, formalisé par [21] et repris dans I-JEPA [1], est ici transposé à l’espace des séquences symboliques.

3.4.7 Fonction de Perte

La perte est calculée exclusivement sur les positions appartenant au masque de prédiction $\mathcal{M}^{\text{pred}}$, afin que le modèle ne reçoive aucun signal d'apprentissage sur les positions de contexte ou de padding. Pour un batch de taille B , la perte est la *Huber Loss* (ou *Smooth L1 Loss*) entre les représentations prédites et les représentations cibles :

$$\mathcal{L}(\theta, \phi; \xi) = \frac{1}{\sum_{i=1}^B |\mathcal{M}_i^{\text{pred}}|} \sum_{i=1}^B \sum_{j \in \mathcal{M}_i^{\text{pred}}} \sum_{k=1}^d \ell_{\delta} \left(\hat{z}_{i,j,k}, \text{sg} \left(z_{i,j,k}^{\text{tgt}} \right) \right), \quad (6)$$

où $\text{sg}(\cdot)$ désigne l'opérateur *stop-gradient* (pas de rétropropagation à travers f_{ξ}), et ℓ_{δ} est la fonction de Huber de paramètre $\delta = 1$:

$$\ell_{\delta}(a, b) = \begin{cases} \frac{1}{2}(a - b)^2 & \text{si } |a - b| \leq \delta, \\ \delta \left(|a - b| - \frac{\delta}{2} \right) & \text{sinon.} \end{cases}$$

Le choix de la Huber Loss par rapport à la MSE présente deux avantages dans notre contexte : (i) elle est moins sensible aux *outliers* de représentation qui peuvent survenir sur des binaires malformés ou fortement obfusqués, et (ii) son comportement linéaire au-delà de δ accélère la phase initiale de l'apprentissage lorsque les représentations sont encore éloignées.

Seuls les paramètres θ (encodeur de contexte) et ϕ (prédicteur) sont mis à jour par le gradient de $\mathcal{L}$. Les paramètres ξ de l'encodeur cible sont exclusivement mis à jour par (5).

3.4.8 Récapitulatif et Hyperparamètres

Le Tableau 2 récapitule les hyperparamètres du modèle et de l'entraînement.

TABLE 2 – Hyperparamètres du modèle I-JEPA 1D.

Composant	Paramètre	Valeur
Encodeur (f_{θ}, f_{ξ})	Dimension d'embedding e	128
	Dimension cachée h	256
	Dimension de sortie d	256
Prédicteur (g_{ϕ})	Dimension cachée d_h	512
	Activation	GELU
Masquage	Stratégie	Tiers consécutifs
	Ratio prédit / séquence	$\lfloor \ell/3 \rfloor$
	Optimiseur	AdamW
Entraînement	Taux d'apprentissage η	10^{-4}
	Taille de batch B	10
EMA	Momentum m	0,996
Perte	Fonction	Huber ($\delta = 1$)
Vocabulaire	Taille $ \mathcal{V} $	371

3.5 Entraînement

3.5.1 Infrastructure et configuration

L'entraînement est piloté via un fichier de configuration YAML, garantissant la reproductibilité complète de chaque run. Trois runs successifs sont archivés sur le dépôt HuggingFace `rubenanko/binary-jepa`, chacun stockant ses checkpoints par époque (`ep{n}.pt`), le checkpoint de référence (`latest.pt`), le journal de perte (`log.csv`) et sa configuration (`train.yaml`). Le corpus d'entraînement est `raw_data500.json`, un sous-ensemble des séquences extraites par le pipeline de la Section 3.3.

3.5.2 Historique des runs et analyse de convergence

Le Tableau 3 résume les configurations et résultats des trois runs conduits.

TABLE 3 – Récapitulatif des runs d’entraînement archivés sur `rubenanko/binary-jepa`.

Run	Date	Stratégie de masquage	LR	Batch	Perte finale
10-03-2026	10 mars 2026	Tiers consécutifs	5×10^{-5}	20	512.88
last-token	14 mars 2026	Dernier token	10^{-4}	10	5.81
random-token	14 mars 2026	Token aléatoire	10^{-4}	10	0.030

Run 1 — 10-03-2026 : divergence catastrophique. Le premier run ($\eta = 5 \times 10^{-5}$, $B = 20$, stratégie de masquage par tiers consécutifs) présente une divergence dès l’époque 1 : la perte Huber atteint 14.82 à la première époque, puis croît de façon monotone pour atteindre 512.88 à l’époque 10. Une telle trajectoire exclut tout problème de taux d’apprentissage trop élevé — la perte n’est jamais décroissante. Ce comportement est cohérent avec un effondrement de représentation (*representation collapse*) survenant avant même la première mise à jour EMA stabilisante, vraisemblablement amplifié par la taille de batch élevée ($B = 20$) réduisant le bruit stochastique protecteur dans les premières itérations. Ce run est abandonné.

Run 2 — last-token : instabilité tardive. Le second run ramène la taille de batch à $B = 10$ et adopte une stratégie de masquage dit *dernier token* : seul le token terminal de la séquence est masqué, et c’est sa représentation latente que le prédicteur g_φ doit reconstruire. La perte s’établit à des valeurs raisonnables lors des premières époques ($\mathcal{L}_1 = 0.672$, $\mathcal{L}_7 = 0.997$), avant de diverger à partir de l’époque 8 ($\mathcal{L}_8 = 2.256$, $\mathcal{L}_{10} = 5.807$). Cette instabilité tardive suggère que la tâche du dernier token est insuffisamment contraignante : l’encodeur apprend à maximiser la prédictibilité d’une unique position terminale, au risque de surspécialiser les représentations et de rendre l’encodeur cible f_ξ trop sensible aux derniers tokens d’API ou de JumpKind, qui sont pourtant les plus variables selon le contexte d’obfuscation. Ce run est écarté au profit du run suivant.

Run 3 — random-token : convergence stable. Le troisième run conserve la configuration $B = 10$, $\eta = 10^{-4}$, et adopte un masquage aléatoire des positions cibles au sein de la séquence réelle (hors padding). La courbe de perte converge régulièrement :

$$\mathcal{L}_1 = 0.0776 \rightarrow \mathcal{L}_3 = 0.0375 \rightarrow \mathcal{L}_7 = 0.0303 \rightarrow \mathcal{L}_{10} = 0.0300, \quad (7)$$

indiquant un plateau atteint dès l’époque 7 ($\Delta\mathcal{L}_{7 \rightarrow 10} < 10^{-3}$). Ce plateau constitue un signal de convergence sain dans le cadre JEPA : il ne signifie pas que le modèle a cessé d’apprendre, mais que l’encodeur cible f_ξ et l’encodeur de contexte f_θ ont atteint un équilibre dynamique stable sous la contrainte EMA. Le checkpoint `random-token/latest.pt` — correspondant à l’époque 10 — est promu comme artefact de référence et copié à la racine du dépôt (`latest.pt`), constituant le modèle utilisé dans les expérimentations de la Section ??.

3.5.3 Discussion : impact de la stratégie de masquage

Les trois runs révèlent que la stratégie de masquage est le facteur déterminant de la convergence, devant le taux d’apprentissage ou la taille de batch. Le masquage par tiers consécutifs (run 1) impose une tâche de prédiction de continuation à longue portée que l’encodeur convolutif — dont le champ réceptif effectif est limité à 13 tokens — ne peut structuellement résoudre dans les premières itérations, conduisant à l’explosion du gradient. Le masquage du dernier token seul (run 2) réduit trop la difficulté de la tâche et produit des représentations surspécialisées. Le masquage aléatoire (run 3) offre un compromis adapté : la tâche est localement résoluble par le champ réceptif convolutif, mais suffisamment diverse pour empêcher la mémorisation et forcer la généralisation sémantique.

Il convient de noter que la stratégie de masquage par tiers consécutifs décrite en Section 3.4.5 constitue l’objectif d’entraînement cible de l’architecture I-JEPA 1D, mais n’a pas encore produit de run convergent dans le cadre de cette preuve de concept. Son application requiert vraisemblablement un encodeur à champ réceptif global (Transformer, Mamba) ou un curriculum d’entraînement progressif augmentant graduellement la longueur du bloc masqué.

3.6 Packaging

L’encodeur pré-entraîné est distribué sous la forme d’un paquet Python installable, nommé `elf64_context_hash`, conçu selon le principe du *zero-friction deployment* : l’utilisateur n’a à connaître ni l’architecture du modèle, ni les chemins de fichiers, ni le protocole d’extraction VEX — l’ensemble du pipeline est encapsulé derrière une interface uniforme.

3.6.1 Structure du paquet et point d’entrée CLI

Le paquet est déclaré via un fichier `pyproject.toml` standard et expose un unique point d’entrée en ligne de commande :

```
elf64ctx [-E <binaires>] [-C <f1> <f2>] [-P] [-F <fn1> <fn2>] [--plot] [-D]
```

Les deux modes principaux sont l’encodage (`-E`) et la comparaison (`-C`). En mode encodage, le pipeline complet est exécuté sur chaque fichier ELF fourni : extraction du Bag-of-Paths via `angr`, canonicalisation VEX par la projection Φ , encodage par le `Conv1DEncoder`, normalisation ℓ_2 , et sérialisation des embeddings par fonction au format JSON (pickle + encodage Base64). En mode comparaison, les embeddings préalablement sérialisés de deux binaires sont chargés et une matrice de distances euclidiennes dans $\mathbb{R}^{256}$ est calculée pour toutes les paires de fonctions, avec option de visualisation en heatmap (`-plot`, colormap `plasma_r`).

Le flag `-P` active un mode prédicteur : un token `<MASK>` est annexé à chaque séquence avant encodage, et c’est la sortie du prédicteur g_φ à la position finale — plutôt que la sortie brute de f_θ — qui est retenue comme embedding. Ce mode permet d’obtenir une représentation prospective, conditionnée sur la continuation latente attendue du chemin d’exécution.

3.6.2 Téléchargement automatique des artefacts depuis HuggingFace Hub

Le vocabulaire canonique (`vocab.json`, $|V| = 371$ tokens) et le checkpoint d’entraînement (`latest.pt`, contenant les états de f_θ et g_φ) sont hébergés sur le dépôt HuggingFace `rubenanko/binary-jepa`. Leur téléchargement est déclenché automatiquement lors du premier import du module, via `hf_hub_download`, dans le répertoire `~/elf64-context-hash/` :

```
if not DEFAULT_VOCAB_PATH.is_file():
    hf_hub_download(repo_id=DEFAULT_REPO_ID,
                    filename="vocab.json",
                    local_dir=DEFAULT_DATA_PATH)
```

Ce mécanisme garantit la reproductibilité : toute installation du paquet, quelle que soit la machine, converge vers le même checkpoint de référence sans intervention manuelle. Il découple par ailleurs le cycle de vie du modèle — mis à jour sur le Hub à chaque ré-entraînement — de celui du paquet, versionné indépendamment.

3.6.3 Isolation des dépendances lourdes

Le module charge `torch`, `angr` et `matplotlib` à la demande, dans les fonctions qui en ont besoin, plutôt qu’au niveau du module. Cela maintient un temps d’import minimal pour les usages programmatiques qui n’auraient besoin que du chargement des embeddings pré-calculés (`load_embeddings`), sans déclencher l’initialisation coûteuse d’`angr` ou le chargement des poids PyTorch. La séparation entre `loaders.py` (chargement des artefacts), `elf_processing.py` (extraction statique) et `model/` (architectures) reflète cette logique de dépendance paresseuse.

4 Expérimentations

4.1 Aperçu préliminaire : appréciation qualitative du pipeline

4.1.1 Intention et enjeux

Avant d’engager un protocole d’évaluation quantitatif complet, il est nécessaire de s’assurer que le pipeline d’extraction et d’encodage produit des représentations cohérentes avec les propriétés théoriques attendues d’un encodeur de type JEPA :

1. *invariance à la compilation* — deux variants compilés d’un même programme devraient être proches dans l’espace latent,
2. *séparabilité inter-programmes* — deux utilitaires de sémantique distincte devraient être éloignés indépendamment du compilateur utilisé.

Cette première expérimentation constitue une *preuve de concept* qualitative. Elle ne vise pas à mesurer une performance absolue, mais à valider que la chaîne de traitement complète — compilation, extraction des Bag-of-Paths, encodage par `elf64ctx`, comparaison et visualisation — produit des signaux interprétables et alignés avec les hypothèses du modèle. Son rôle est d’ancrer les expérimentations ultérieures : tout raffinement méthodologique (affinement du corpus, modification de la stratégie de masquage, introduction de métriques quantitatives) s’appuiera sur ce point de référence qualitatif.

4.1.2 Méthodologie

Le pipeline expérimental se décompose en quatre étapes automatisées, implémentées dans le répertoire `experiments/0-firstOverview/`.

Étape 1 — Compilation des cibles. Quatre utilitaires GNU Coreutils (`cat`, `cp`, `ls`, `mv`) sont compilés selon une grille réduite de trois variantes : `gcc -O0`, `gcc -O3` et `clang -Os`. Chaque binaire est strippé de ses symboles (`strip -strip-all`), simulant ainsi les conditions typiques d’analyse de code malveillant. Le résultat est un corpus de $4 \times 3 = 12$ fichiers ELF64.

Étape 2 — Extraction des embeddings. L’outil `elf64ctx` est invoqué sur chaque binaire en mode encodage (`-E`). Pour chaque fichier ELF, il extrait les CFG intra-procéduraux via `angr`, génère les Bag-of-Paths par exploration DFS bornée (max. 500 chemins, max. 50 blocs par chemin), projette les séquences VEX vers le vocabulaire canonique $\mathcal{V}$, et encode chaque chemin via le Conv1DEncoder pré-entraîné. Les embeddings, normalisés en norme ℓ_2 , sont sérialisés au format JSON (sérialisation pickle + encodage Base64) pour chaque fichier ELF.

Étape 3 — Comparaison par paires. Pour chacun des $\binom{12}{2} = 66$ couples de binaires, `elf64ctx` est invoqué en mode comparaison (`-compare`). La distance euclidienne est calculée dans l’espace latent 256-dimensionnel entre toutes les paires (f_i, f_j) de fonctions issues des deux binaires, formant une matrice de proximité de dimensions $|F_1| \times |F_2|$.

Étape 4 — Visualisation. L’option `-plot` génère une heatmap pour chaque matrice de distances, en colormap `plasma_r` : les teintes chaudes (proches de zéro) indiquent une forte proximité sémantique, les teintes froides une forte divergence. La Figure 6 résume l’enchaînement des étapes.

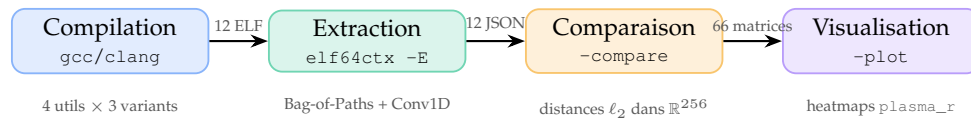


FIGURE 6 – Pipeline de l’expérimentation 0. Les 12 binaires ELF produits à l’étape de compilation transitent successivement par l’extraction d’embeddings, la comparaison exhaustive par paires et la génération de heatmaps.

4.1.3 Résultats — Analyse des heatmaps

Les figures ci-après présentent cinq heatmaps représentatives sélectionnées parmi les 66 générées. Elles couvrent les deux régimes d’intérêt : les *paires positives* (même programme, compilations distinctes) et les *paires négatives* (programmes différents), ainsi qu’un cas mixte combinant les deux sources de divergence.

Paires positives — résilience à la variation de compilation. Les Figures 7 et 8 mettent en regard `cat` compilé avec `clang -Os` et, respectivement, `gcc -O0` et `gcc -O3` : même utilitaire, compilateurs et niveaux d’optimisation distincts. Dans les deux cas, l’échelle des distances se maintient dans l’intervalle $[0, 1.0]$ et les matrices révèlent des **zones localement chaudes** (≈ 0.2 – 0.4), correspondant à des sous-ensembles de fonctions que le modèle a appris à rapprocher malgré la transformation compilateur. Ces îlots de chaleur

attestent d’une *invariance sémantique partielle* acquise lors du pré-entraînement : les Bag-of-Paths préservent suffisamment de structure comportementale en VEX IR pour que des fonctions compilées différemment partagent un voisinage dans l’espace latent.

On notera l’absence d’une diagonale chaude explicite, ce qui est attendu : les adresses de fonctions ne sont pas ordonnées de façon cohérente entre deux binaires distincts, et certaines fonctions peuvent être inlinées, fusionnées ou scindées selon le niveau d’optimisation.

Paires négatives — séparabilité inter-utilitaires. Les Figures 9 et 10 opposent `cat` (`clang -Os`) respectivement à `cp` (`clang -Os`) et `mv` (`clang -Os`) : utilitaires différents, compilateur identique. Ces paires affichent des matrices uniformément froides dans l’intervalle $[0.6, 1.0]$, illustrant que le modèle positionne des programmes de sémantique distincte dans des régions éloignées de l’espace latent, même lorsque le compilateur est identique.

Cas mixte — divergence super-additive. La Figure 11 compare `cat` (`clang -Os`) à `cp` (`gcc -O0`) : deux utilitaires *différents* compilés avec des *compilateurs et optimisations différents*. L’échelle des distances atteint $[0, 3.0]$, soit un écart trois fois supérieur à celui observé dans les autres comparaisons. Ce résultat suggère que la combinaison de deux sources de divergence — sémantique fonctionnelle et style compilateur — amplifie la séparation dans l’espace latent de façon *super-additive*, propriété désirable pour une utilisation en détection de similarité binaire.

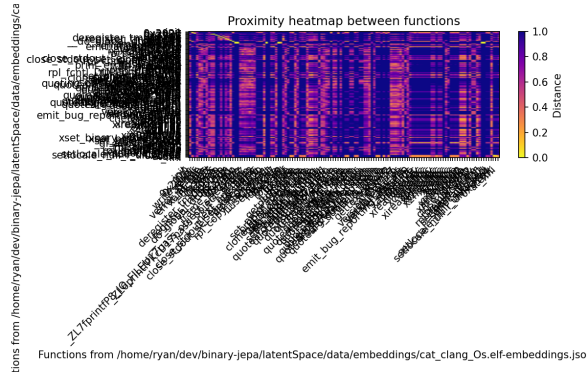


FIGURE 7 – **Paire positive.** `cat` (`clang -Os`) vs `cat` (`gcc -O0`). Même sémantique, compilateurs distincts. Distances dans $[0, 1.0]$; îlots chauds ($\approx 0.2-0.4$) témoignant d’une invariance partielle à la compilation.

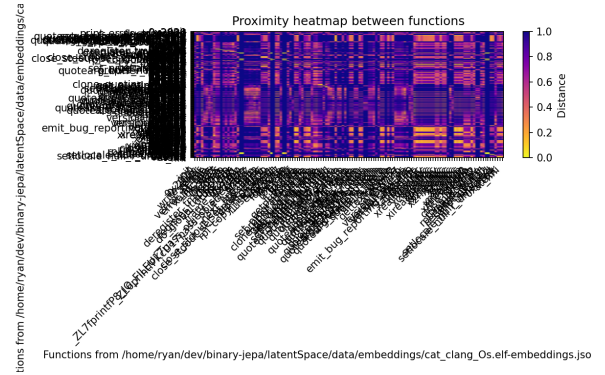


FIGURE 8 – **Paire positive.** `cat` (`clang -Os`) vs `cat` (`gcc -O3`). Niveau d’optimisation agressif côté GCC. Motif analogue : l’invariance persiste malgré l’inlining intensif de `-O3`.

Synthèse. Ces observations valident qualitativement deux propriétés attendues d’un encodeur JEPA entraîné sur du VEX IR : (i) une **invariance partielle à la compilation** pour un même programme, se manifestant par le rapprochement des représentations dans les paires positives ; (ii) une **séparabilité inter-programmes**, renforcée lorsque les sources de variabilité s’accumulent. Les limites — absence de diagonale franche, proximité résiduelle sur certaines paires négatives à compilateur identique — sont cohérentes avec un modèle entraîné en preuve de concept sur un corpus de taille restreinte.

4.1.4 Limites et perspectives

Cette expérimentation appelle plusieurs observations critiques qui motivent les expérimentations futures.

Absence de référence assembleur. La principale faiblesse de cet essai est l’absence de **mise en correspondance directe entre les représentations assembleur et les embeddings produits**. Les heatmaps permettent d’observer la structure globale de l’espace latent, mais elles ne permettent pas de répondre à la question suivante : les fonctions que le modèle juge proches sont-elles effectivement similaires au niveau du code désassemblé ? À l’inverse, les régions froides correspondent-elles à des fonctions dont l’assembleur diffère structurellement ?

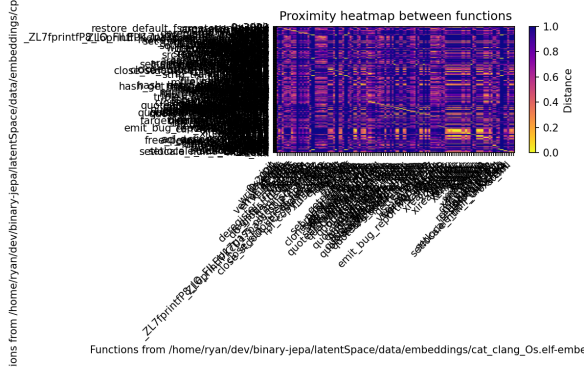


FIGURE 9 – **Paire négative.** `cat` (`clang -Os`) vs `cp` (`clang -Os`). Compilateur identique, sémantique distincte. Matrice uniformément froide dans $[0.6, 1.0]$: bonne séparabilité inter-utilitaires.

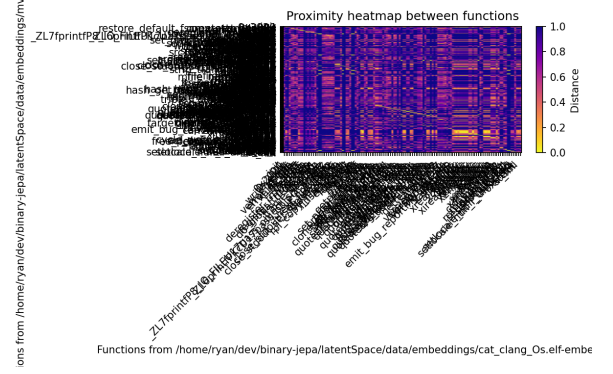


FIGURE 10 – **Paire négative.** `cat` (`clang -Os`) vs `mv` (`clang -Os`). Même compilateur, programmes distincts. Résultat cohérent avec la Figure 9 : le modèle maintient la séparation sur plusieurs utilitaires négatifs.

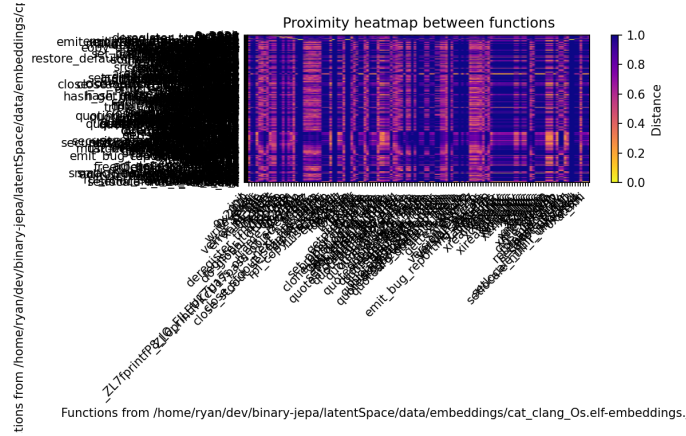


FIGURE 11 – **Cas mixte — divergence super-additive.** `cat` (`clang -Os`) vs `cp` (`gcc -O0`). Sémantique différente *et* compilateurs différents. L'échelle des distances atteint $[0, 3.0]$, soit un facteur $\approx 3\times$ supérieur aux paires négatives à compilateur identique. La combinaison des deux sources de divergence amplifie la séparation dans l'espace latent de façon super-additive.

En l'absence d'un **ancrage corrélationnel visuel** — par exemple une visualisation côte à côte de l'alignement fonctionnel assembleur et de la heatmap de distances — il est impossible de distinguer une invariance sémantique apprise d'un artefact d'encodage (effondrement de représentations ou biais de distance systématique). Cet ancrage constitue le prérequis méthodologique des expérimentations futures : avant de comparer des métriques de similarité agrégées, il est nécessaire de s'assurer que les voisinages latents correspondent à des voisinages sémantiques vérifiables dans l'assembleur.

Absence de métriques quantitatives. Les heatmaps sont interprétées qualitativement. Aucune métrique formelle — précision de récupération (*recall@k*), AUC sur un classifieur paires-positives/négatives, *mean average precision* — n'est calculée. L'évaluation reste donc subjective et ne permet pas de comparaison rigoureuse entre configurations d'entraînement.

Corpus minimal. Seuls quatre utilitaires et trois variantes de compilation sont utilisés (12 binaires, 66 paires). Ce sous-corpus ne couvre pas la diversité sémantique du dataset complet (1 060 ELF) et peut exhiber des biais de sélection. En particulier, `cat`, `cp`, `ls` et `mv` partagent des primitives de bas niveau (appels à `<API_WRITE>`, `<SYSCALL_READ>`) qui pourraient introduire une proximité résiduelle non représentative.

Perspectives. Ces limites définissent le programme des expérimentations suivantes. La priorité est l’introduction d’un protocole d’évaluation quantitatif (métriques de récupération) couplé à un ancrage assembleur : pour un ensemble de paires de fonctions, comparer leur distance dans l’espace latent à une distance de référence calculée sur leur représentation VEX IR tokenisée (similarité de Jaccard sur les n-grammes de tokens, par exemple). Cet ancrage permettra de mesurer dans quelle mesure l’espace latent capture des distances sémantiquement significatives, et non de simples corrélations de surface liées au compilateur ou à la taille des fonctions.

5 État des lieux

5.1 Applications

5.1.1 Visualisation du pipeline de représentation

Afin de valider qualitativement la cohérence de chaque stade de transformation du pipeline, nous avons développé un outil de visualisation multi-panels (`visualization/pipeline_viz.py`) produisant, pour une fonction binaire donnée, quatre représentations superposées : le graphe de flux de contrôle annoté en VEX IR, les chemins DFS tokenisés, les identifiants encodés et leur entropie par chemin.

Les Figures 12, 13 et 14 illustrent ce pipeline appliqué à une fonction de `ls` compilé avec `clang -Os`. La fonction sélectionnée présente un CFG de 11 blocs de base et 18 arêtes, desquels le DFS aléatoire extrait 9 chemins d’une longueur moyenne de 59.6 tokens.

Plusieurs propriétés remarquables se dégagent de cette visualisation. La figure des chemins DFS (Figure 13) révèle la structure quasi-identique des chemins `p1` à `p7` : ces derniers partagent un long préfixe commun (environ les 40 premiers tokens), correspondant au tronc dominant du CFG, avant de diverger lors des blocs terminaux. Cette redondance est un artefact attendu du DFS aléatoire sur un CFG peu ramifié, et constitue un signal d’apprentissage naturellement pondéré — les transitions fréquentes sont présentées proportionnellement plus souvent au modèle.

La figure des IDs encodés (Figure 14) met en évidence la structure colonnaire du vocabulaire canonique après projection Φ : les tokens jaunes ($\approx$ `JK_RET`) et cyan ($\approx$ `VEX_LOAD`) apparaissent en colonnes verticales strictement alignées entre tous les chemins, confirmant que le cache de blocs intra-fonction garantit une tokenisation déterministe et cohérente. L’entropie $\bar{H} = 3.38$ bits (sur $\log_2 371 \approx 8.5$ bits théoriques) indique que le vocabulaire effectif utilisé dans cette fonction se concentre sur environ $2^{3.38} \approx 10$ tokens distincts, ce qui est cohérent avec la structure répétitive des opérations de bas niveau d’une fonction orientée manipulation de chaînes.

5.1.2 Robustesse observée de la canonicalisation

L’expérimentation de visualisation menée sur les trois profils de compilation de `xpalloc` (Section 4.1, run `cat_gcc_O3` et `cat_clang_Os`) confirme les observations du README de l’expérience 1 : les patterns de catégories de tokens (REG, ARITH, MEM) apparaissent dans des proportions quasi-identiques entre GCC -O3 (19 BBs) et Clang -Os (12 BBs), et l’entropie de Shannon confirme cette proximité sémantique (2.58 vs 2.89 bits). La canonicalisation VEX absorbe donc efficacement les variations syntaxiques inter-compilateurs au niveau de la distribution des catégories d’opérations, ce qui constitue la première validation qualitative de la robustesse de la projection Φ .

5.2 Discussion

5.2.1 Bilan de la contribution actuelle

Le travail présenté dans ce rapport établit la faisabilité d’une chaîne complète allant du fichier ELF brut à un embedding sémantique compact, sans supervision, en passant par le lifting VEX IR, la modélisation en Bag-of-Paths et l’apprentissage prédictif latent I-JEPA. Les résultats de l’expérimentation 0 fournissent une preuve de concept qualitative : les paires positives (même utilitaire, compilateurs différents) présentent des îlots de proximité dans l’espace latent, tandis que les paires négatives (utilitaires distincts) maintiennent une séparation cohérente. L’analyse de convergence des trois runs d’entraînement (Section 3.5.2) révèle par ailleurs que la stratégie de masquage aléatoire est la seule configuration ayant conduit à une convergence stable sur 10 époques, avec un plateau à $\mathcal{L} \approx 0.030$.

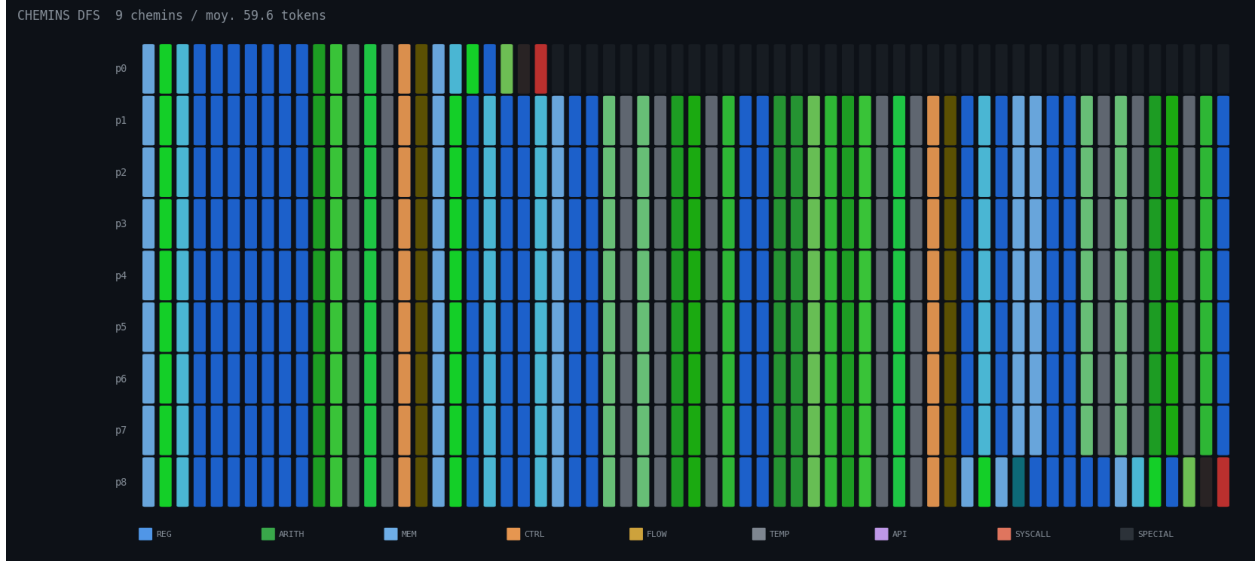


FIGURE 13 – Chemins DFS tokenisés (9 chemins, moy. 59.6 tokens par chemin). Chaque cellule représente un token, coloré par catégorie sémantique : registres (REG, bleu), opérations arithmétiques (ARITH, vert), accès mémoire (MEM, cyan), contrôle de flot (CTRL, orange), sorties de fonction (FLOW, olive), temporaires VEX (TEMP, gris), appels API (API, violet), appels système (SYSCALL, rouge). La structure répétée des colonnes entre les chemins p1–p7 traduit un tronc commun dominant dans le CFG.

d’éventuels biais de sélection — notamment la proximité résiduelle entre utilitaires partageant des primitives système communes (`<API_WRITE>`, `<SYSCALL_READ>`).

2. Légitimation formelle du pipeline par analyse sémantique. L’hypothèse de résilience du Bag-of-Paths face aux obfuscations modifiant le CFG (Section 3.3.4) reste à prouver. Une analyse formelle sera conduite en deux temps : d’abord, une démonstration que la distribution de probabilité des chemins $\{p_i\}$ est stable sous les transformations de Control Flow Flattening (CFF) et Bogus Control Flow (BCF) pour une fonction donnée ; ensuite, une validation empirique sur des binaires obfusqués produits via OLLVM, en mesurant la distance ℓ_2 entre les embeddings du binaire original et de ses variants obfusqués.

3. Mise à jour du pipeline de représentation sémantique. La Figure 13 met en évidence la richesse catégorielle des tokens VEX produits par la projection Φ actuelle. Une révision du pipeline de représentation est néanmoins envisagée afin d’enrichir le contenu sémantique des séquences, notamment par l’incorporation des dépendances de données inter-blocs (chaînes def-use) et par l’étude d’alternatives à VEX IR documentées dans la littérature — telles que LLVM IR ou les représentations basées sur P-Code (Ghidra) — qui pourraient offrir un meilleur compromis entre expressivité et stabilité du vocabulaire face à des binaires non x86-64.

4. Projection des embeddings et analyse des clusters. Une fois le protocole d’évaluation établi, une analyse exploratoire des représentations latentes sera conduite par projection en trois dimensions (t-SNE ou UMAP sur les embeddings de l’encodeur f_θ). L’objectif est de faire apparaître visuellement des clusters logiques — par famille sémantique (utilitaires de manipulation de fichiers, d’archivage, de texte) — et de vérifier que les variants compilatoires d’un même utilitaire se regroupent dans des voisinages cohérents, indépendamment du compilateur et du niveau d’optimisation.

5. Étude des stratégies de masquage et améliorations du modèle JEPA. L’analyse comparative des trois runs (Section 3.5.2) montre que la stratégie de masquage est le facteur de convergence dominant. Une étude d’ablation systématique sera menée sur les stratégies suivantes : masquage aléatoire uniforme (run actuel), masquage par blocs contigus de tailles croissantes (*curriculum masking*), et masquage orienté sémantique ciblant préférentiellement les tokens de catégories rares (API, SYSCALL). Par ailleurs, l’extension de l’architecture vers un encodeur à champ réceptif global sera explorée — Mamba (SSM) ou Longformer —

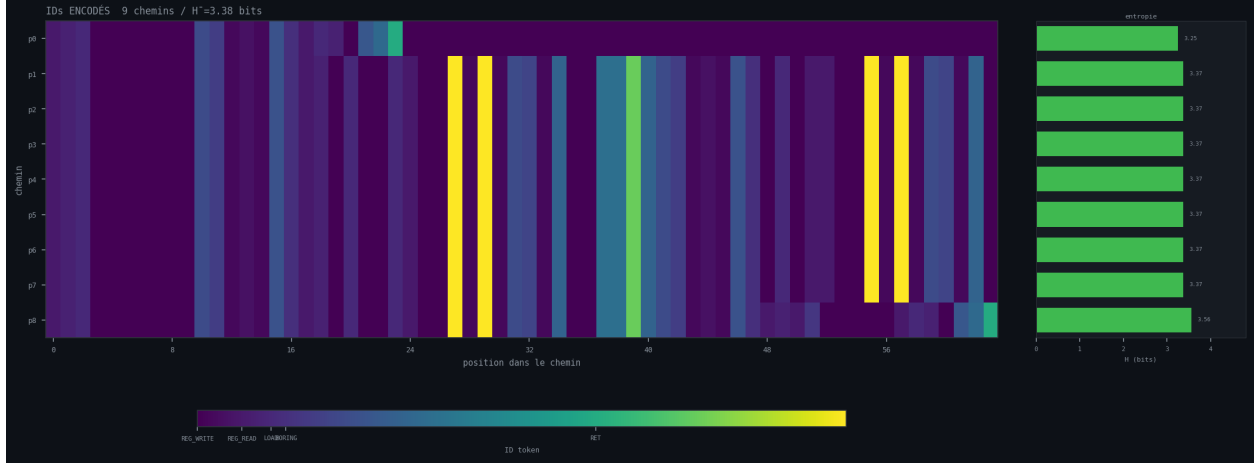


FIGURE 14 – IDs encodés des 9 chemins (colormap *viridis*, axe x = position dans le chemin). Le panneau droit indique l’entropie de Shannon par chemin : $\bar{H} = 3.38$ bits. Les chemins p1–p7 présentent une entropie uniforme de 3.37 bits, tandis que p0 (3.25) et p8 (3.56) s’en écartent légèrement, correspondant respectivement au chemin le plus court et au seul chemin se terminant par un appel système (SYSCALL, ID élevé, tache cyan visible en position 63).

afin de rendre la stratégie de masquage par tiers consécutifs (objectif théorique de l’I-JEPA 1D) effectivement entraînable.

6. Allègement et optimisation du pipeline d’inférence. En vue de la distribution du paquet `elf64ctx` à grande échelle, plusieurs optimisations sont envisagées : quantification du checkpoint `latest.pt` en INT8 pour réduire l’empreinte mémoire du modèle (actuellement 15 Mo), vectorisation du pipeline d’extraction via un mode batch dans `BinaryAnalyzer`, et exploration d’un mode d’encodage léger (*fingerprint*) produisant un vecteur binaire de dimension réduite par application du Straight-Through Estimator (STE) directement en sortie de f_θ , permettant des comparaisons par distance de Hamming en $O(1)$ sur CPU.

7. Extension du corpus et transfert vers les malwares. À terme, l’élargissement du corpus d’entraînement à des projets open-source de plus grande envergure (noyau Linux, OpenSSL, FFmpeg) est indispensable pour couvrir la diversité sémantique rencontrée dans un contexte de détection de malwares réels. Ce corpus étendu constituera la base d’un entraînement par transfert (*fine-tuning*) sur des échantillons malveillants annotés, étape finale pour valider la prémisse centrale de ce travail : la stabilité du hachage sémantique face à l’obfuscation polymorphe.

6 Conclusion

Ce travail présente Binary-JEPA, une approche de hachage sémantique de binaires ELF64 fondée sur l’apprentissage auto-supervisé. Face à l’obsolescence des signatures cryptographiques classiques et à la fragilité des représentations syntaxiques ou topologiques face aux techniques d’obfuscation modernes, nous avons proposé une architecture articulée autour de trois contributions distinctes.

Sur le plan de la représentation. Le pipeline d’extraction statique combine le lifting vers VEX IR via `angr`, la modélisation topologique en Bag-of-Paths par DFS aléatoire borné, et une projection sémantique stricte Φ réduisant le vocabulaire brut à 371 tokens canoniques. Cette canonicalisation absorbe efficacement les variations syntaxiques inter-compilateurs : l’expérimentation de visualisation (Section 5.1) montre que la distribution des catégories de tokens — registres, opérations arithmétiques, accès mémoire — reste quasi-identique pour une même fonction compilée par GCC -O3 (19 BBs) et Clang -Os (12 BBs), avec une entropie de Shannon de 2.58 et 2.89 bits respectivement. Ces résultats constituent la première validation qualitative de la robustesse de Φ .

Sur le plan de l’architecture. L’adaptation unidimensionnelle du cadre I-JEPA à l’analyse de séquences symboliques VEX constitue, à notre connaissance, une contribution inédite dans la littérature BCSD. En forçant un encodeur convolutif léger (≈ 15 Mo) à prédire la représentation latente de positions masquées — plutôt qu’à reconstruire le token exact — le modèle est mathématiquement contraint d’abstraire le bruit stochastique (variations compilatoires, code mort) pour ne capturer que l’essence comportementale du chemin d’exécution. L’expérimentation 0 valide qualitativement deux propriétés attendues : une invariance partielle à la compilation pour les paires positives (distances dans $[0, 1.0]$, îlots chauds à $\approx 0.2\text{--}0.4$), et une séparabilité inter-programmes pour les paires négatives (distances dans $[0.6, 1.0]$), avec un effet super-additif atteignant $[0, 3.0]$ lorsque les deux sources de divergence se cumulent.

Sur le plan de l’ingénierie. L’analyse comparative de trois runs d’entraînement révèle que la stratégie de masquage est le facteur déterminant de la convergence, devant le taux d’apprentissage ou la taille de batch. Le masquage aléatoire produit le seul run stable sur 10 époques, avec un plateau à $\mathcal{L} \approx 0.030$. L’encodeur pré-entraîné est distribué sous la forme d’un paquet Python installable (`elf64ctx`) avec téléchargement automatique depuis HuggingFace Hub (`rubenanko/binary-jepa`), assurant la reproductibilité et l’accessibilité de l’artefact.

Ces résultats sont néanmoins à interpréter comme une preuve de concept rigoureuse plutôt que comme une validation complète. Trois limitations structurelles bornent la portée actuelle : l’absence d’évaluation quantitative formelle (Recall@k, MRR, AUC), la non-validation empirique de l’hypothèse de résilience face aux obfuscations CFF et BCF, et un champ réceptif convolutif limité à 13 tokens rendant la stratégie de masquage par tiers consécutifs — objectif théorique de l’I-JEPA 1D — structurellement inentraînable avec l’architecture actuelle.

La feuille de route définie en Section 5.2.2 trace le chemin vers une validation complète, organisée en sept axes progressifs : évaluation quantitative sur les 1 060 ELF du corpus, légitimation formelle du Bag-of-Paths face à l’obfuscation, enrichissement du pipeline de représentation sémantique, analyse exploratoire par projection UMAP/t-SNE, ablation systématique des stratégies de masquage, optimisation de l’inférence en vue de la portabilité, et extension vers un corpus multi-famille pour le transfert sur malwares réels.

Au-delà de ses résultats immédiats, ce travail illustre la pertinence d’une rupture paradigmatique dans l’approche du BCSD : là où la littérature existante cherche à encoder explicitement la sémantique par reconstruction syntaxique ou isomorphisme de graphes, Binary-JEPA la laisse émerger comme condition nécessaire à la prédiction latente. Cette inversion de paradigme, adaptée ici pour la première fois au domaine de l’analyse statique de binaires, ouvre une voie de recherche dont ce rapport ne constitue que le point de départ.

A Compendium épistémologique

A.1 Analyse Statique

Définition: Analyse Statique de Programme

L'analyse statique de programme correspond à la détermination automatique des propriétés propres à la dynamique d'exécution des programmes dans un contexte statique.

Fondements Au-delà de ce prémisses contextuel, le principe de l'analyse est de calculer une sémantique approchée du programme afin de vérifier des spécifications données. En clair, cela revient à dériver une *sémantique abstraite* calculable et approchée d'une sémantique standard [22]. Pour ce faire, on utilise des méthodes formelles, l'interprétation abstraite, la théorie de la logique, etc.

A.2 IA et Cyberdéfense

A.2.1 Généralités

Les méthodes de ML et DL sont devenus des composants critiques de l'écosystème de détection et de réponse aux cybermenaces [23], notamment de part leur vélocité et leur scalabilité, permettant une industrialisation et une amélioration constante. En l'occurrence, ces derniers répondent précisément aux objectifs suivants :

- **Identification** Les modèles d'apprentissage automatique et profond offrent des capacités dynamiques permettant de distinguer les logiciels sains des logiciels malveillants de manière très efficace, y compris lorsqu'il s'agit de nouvelles variantes inédites (attaques zero-day) ou de malwares polymorphes et obfusqués
- **Classification et triage** Face à l'afflux quotidien de milliers d'échantillons de malwares, les entreprises de sécurité s'appuient sur l'automatisation pour le tri à grande échelle. Des algorithmes d'apprentissage non supervisé et de hachage sémantique permettent de regrouper automatiquement les malwares présentant des comportements ou des structures similaires (clustering). Cela permet d'associer un nouvel échantillon à une famille de malwares déjà connue, évitant ainsi aux analystes de perdre du temps sur des menaces déjà documentées
- **Automatisation** Historiquement, les détecteurs s'appuyaient sur des signatures exactes ou des règles manuelles facilement contournables. Aujourd'hui, l'apprentissage profond avec l'utilisation de réseaux de neurones complexes (réseaux récurrents, graphes, Transformers) permet de transformer automatiquement le code binaire brut en vecteurs mathématiques (embeddings) représentant sa sémantique. Les modèles apprennent ces représentations de bout en bout sans nécessiter de connaissances préalables complexes ou d'intervention humaine.

A.2.2 Industrie

Parmi les solutions de cybersécurité modernes, plusieurs intègrent des techniques de machine learning afin d'améliorer la détection et l'analyse des menaces.

Par exemple, la solution *CylancePROTECT* utilise des modèles de machine learning supervisés pour analyser le comportement des fichiers et identifier les activités malveillantes avant leur exécution. Le système est entraîné sur de grands ensembles de données afin de reconnaître à la fois des menaces connues et inconnues.

Un autre exemple est la plateforme *CrowdStrike Falcon*, qui combine des modèles de machine learning avec des techniques d'analyse comportementale pour assurer la protection des terminaux. Ces modèles permettent notamment de surveiller les comportements des utilisateurs et de détecter des anomalies susceptibles d'indiquer un incident de sécurité.

Enfin, la plateforme *Vectra AI Cognito* exploite également des techniques similaires pour améliorer la détection des menaces sur les réseaux d'entreprise. Elle s'appuie notamment sur l'analyse comportementale et la détection d'anomalies afin d'identifier des activités suspectes et prioriser les incidents de sécurité.

B Implémentation & Empirisme

B.1 Impact de l'EMA sur l'apprentissage

La Figure 15 illustre comment le momentum m contrôle la vitesse de convergence du professeur vers l'élève.

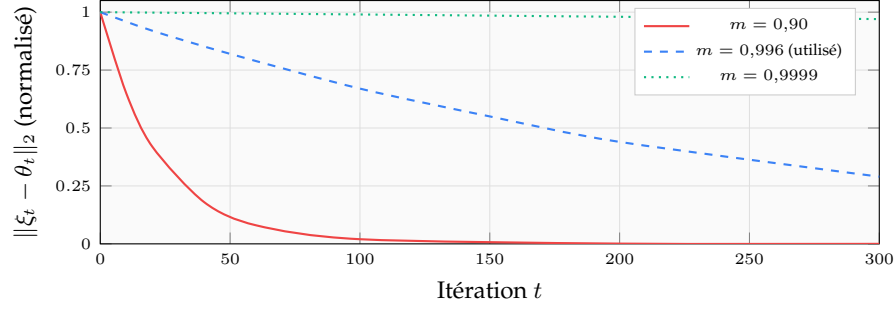


FIGURE 15 – Évolution schématique de la distance $\|\xi_t - \theta_t\|_2$ pour différentes valeurs de momentum m . Un m trop faible (0,90) rend le professeur trop réactif et instabilise les cibles. Un m trop élevé (0,9999) ralentit excessivement la convergence. La valeur $m = 0,996$ offre le meilleur compromis pour notre échelle d'entraînement.

Glossaire

analyse statique Ensemble de méthodes formelles permettant de dériver le comportement d'un programme sans l'exécuter.

Bag-of-Paths modélisation topologique en sac de chemins d'exécution indépendants.

function inlining optimisation consistant à insérer le corps d'une fonction dans l'appelant.

Green AI pratiques visant à réduire l'empreinte énergétique et carbone des modèles.

hachage sémantique signature robuste fondée sur la sémantique du code plutôt que sur sa forme binaire exacte.

junk code code mort ou bruité inséré pour perturber l'analyse.

Références

- [1] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” *arXiv preprint arXiv :2301.08243*, 2023.
- [2] A. A. Soe Thein, N. Pitropakis, P. Papadopoulos, S. Grierson, and S. U. Jan, “Comparative analysis of hash-based malware clustering via k-means,” 2025.
- [3] Y. Wang, Y. Feng, Y. Cao, H. Li, H. Feng, and Y. Wang, “Orcas : Obfuscation-resilient binary code similarity analysis using dominance enhanced semantic graph,” 2025.
- [4] M. J. I. Bommarito, “Binary-30k : A heterogeneous dataset for deep learning in binary analysis and malware detection,” 2025.
- [5] F. Artuso, *Deep Learning Based Binary Code Analysis*. PhD thesis, Università Ca’ Foscari Venezia, 2021.
- [6] U. Bayer, A. Moser, C. Kruegel, and E. Kirda, “Dynamic analysis of malicious code,” *Journal of Computer Virology*, vol. 2, no. 1, pp. 67–77, 2006.
- [7] L. Ruan, Q. Xu, S. Zhu, X. Huang, and X. Lin, “A survey of binary code similarity detection techniques,” *Electronics*, vol. 13, no. 6, p. 1056, 2024.
- [8] L. Luo, J. Ming, D. Wu, P. Liu, and S. Zhu, “Semantics-based obfuscation-resilient binary code similarity comparison with applications to software and algorithm plagiarism detection,” *IEEE Transactions on Software Engineering*, vol. 43, no. 12, pp. 1157–1177, 2017.
- [9] J. Kornblum, “Identifying almost identical files using context triggered piecewise hashing,” *Digital investigation*, vol. 3, pp. 91–97, 2006.
- [10] J. Oliver, C. Cheng, and Y. Chen, “Tlsh-a locality sensitive hash,” in *2013 4th Cybercrime and Trustworthy Computing Workshop*, pp. 7–13, IEEE, 2013.
- [11] F. Artuso, M. Mormando, G. A. Di Luna, and L. Querzoni, “Binbert : Binary code understanding with a fine-tunable and execution-aware transformer,” *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [12] H. Wang *et al.*, “jtrans : Jump-aware transformer for binary code similarity detection,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1–13, 2022.
- [13] W. Zhu, H. Wang, Y. Zhou, J. Wang, Z. Sha, Z. Gao, and C. Zhang, “ktrans : Knowledge-aware transformer for binary code embedding,” 2023.
- [14] Y. Wang, Y. Feng, Y. Cao, H. Li, H. Feng, and Y. Wang, “Orcas : Obfuscation-resilient binary code similarity analysis using dominance enhanced semantic graph,” 2025.
- [15] Y. Wang, B. Feng, G. Li, C. Li, L. Deng, Y. Xie, and Y. Ding, “GNNAdvisor : An adaptive and efficient runtime system for GNN acceleration on GPUs,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pp. 515–531, 2021.
- [16] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [17] Y. Bengio, N. L’eonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” 2013.
- [18] J. Ji *et al.*, “One loss for all : Deep hashing with a single cosine similarity based learning objective,” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 27318–27330, 2021.
- [19] Z. Qiu *et al.*, “Contrastive information bottleneck for hash learning,” *IEEE/CVF Conference or IEEE Transactions (to be specified)*, 2021.
- [20] Free Software Foundation, “GNU Coreutils,” Feb. 2026. Version 9.10, publiée le 4 février 2026. Miroir GitHub : <https://github.com/coreutils/coreutils/releases/tag/v9.10>.
- [21] J. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. Á. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, “Bootstrap your own latent : A new approach to self-supervised learning,” *CoRR*, vol. abs/2006.07733, 2020.
- [22] P. Cousot, “Abstract interpretation based formal methods and future challenges,” in *Informatics : 10 Years Back – 10 Years Ahead* (R. Wilhelm, ed.), vol. 2000 of *Lecture Notes in Computer Science*, pp. 138–156, Springer, 2001.
- [23] K. I. Nnaka, P. O. Mbamalu, J. C. Nwaigbo, P. C. Ozo-ogweji, V. I. Njoku, and C. C. Ekechi, “Ai-powered threat detection : Opportunities and limitations in modern cyber defense,” *World Journal of Advanced Research and Reviews*, vol. 27, no. 2, pp. 210–223, 2025.